شتاب‌دهنده‌های سخت‌افزاری
دانشکده مهندسی و علوم کامپیوتر
بهار ۱۴۰۳-۱۴۰۴
استاد درس: دارا رحمتی
طراحی تمرین: زهرا تاکی و متین فیروزبخت

تمرین چهارم

تاریخ انتشار : ۱۴۰۴/۰۲/۱۳ - تاریخ تحویل : ۱۴۰۴/۰۲/۲۳

معرفی

هدف ار این تمرین کوانتزاسیون[1] و هرس کردن[2] یک شبکه عصبی پیچشی[3] با استفاده از روش‌های موجود می‌باشد. همانطور که میدانید شبکه های عصبی پیچشی یکی ار مهم ترین شبکه ها در یکی از مهم ترین کاربردها یعنی تصاویر هستند. این شبکه ها با داشتن لایه هایی، عملیات کانولوشن را روی داده های دوبعدی انجام میدهد و در وظایف دسته‌بندی[4] ، شناسایی[5] و ... نقش مهمی دارند. همچنین هرینه بالای محاسبات مربوط به این شبکه ها نیر باعث شده تا مسئله شتابدهی آنها اهمیت بالایی داشته باشد.

کوانتزاسیون چیست و چرا از آن استفاده می‌کنیم؟

کوانتزاسیون یکی از روش‌های مهم در بهینه‌سازی مدل‌های یادگیری عمیق است که با هدف کاهش پیچیدگی محاسباتی و مصرف حافظه مورد استفاده قرار می‌گیرد. اغلب شبکه‌های عصبی عمیق، به‌ویژه مدل‌های پیشرفته مانند ترنسفورمرها[6]، دارای میلیاردها پارامتر هستند. به عنوان مثال، مدل معروف GPT-3 که در حوزه پردازش زبان طبیعی[7] کاربرد دارد، شامل حدود ۱۷۵ میلیارد پارامتر است. اگر هر پارامتر با دقت ۳۲ بیت ذخیره شود، با یک محاسبه ساده می‌توان دریافت که فضای مورد نیاز برای ذخیره این مدل حدود ۷۰۰ گیگابایت خواهد بود:

$$\frac{175 \times 10^9 \times 32}{8 \times 10^9} = 700\ GB$$

ذخیره چنین حجمی از داده در حافظه برای انجام عملیات استنتاج[8] در دستگاه‌های معمولی مانند رایانه‌های شخصی یا تلفن‌های همراه بسیار دشوار است. به همین دلیل، استفاده از روش‌هایی مانند کوانتزاسیون برای کاهش حجم مدل، بدون افت محسوس در دقت آن، ضروری به نظر می‌رسد.

از منظر محاسباتی نیز، اجرای عملیات روی اعداد صحیح برای سخت‌افزارهای دیجیتال بسیار سریع‌تر و ساده‌تر از اجرای همان عملیات بر روی اعداد اعشاری است. همان‌طور که در شکل ۱ مشاهده می‌شود، مشابه انسان‌ها، رایانه‌ها نیز در محاسبه اعداد

---

[1] . Quantization
[2] . Pruning
[3] . Convolutional Neural Network (CNN)
[4] . Classification
[5] . Detection
[6] . Transformers
[7] . Natural Language Processing
[8] . Inference

اعشاری نسبت به اعداد صحیح عملکرد کندتری دارند. برای مثال، ضرب دو عدد صحیح مانند 2×2 بسیار سریع انجام می‌شود، اما انجام ضرب بین دو عدد اعشاری مانند 2.0121×1.0921 پیچیده‌تر بوده و منابع محاسباتی بیشتری نیاز دارد.

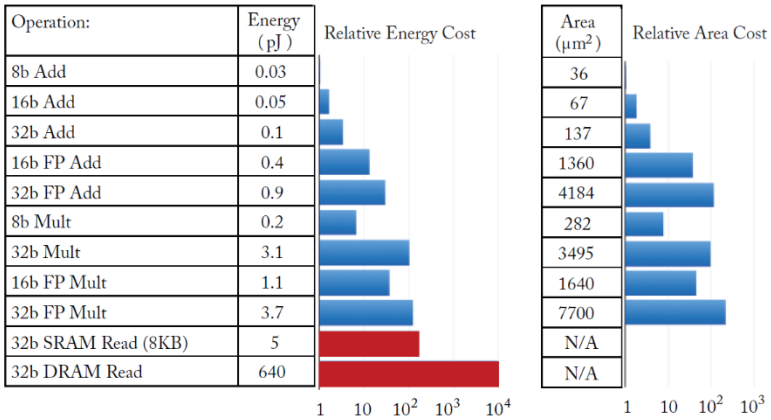

Figure 1 - The area and energy cost for additions and multiplications at different precision [1]

یکی از راهکارهای مؤثر برای کاهش نیاز به منابع محاسباتی و حافظه در شبکه‌های عصبی عمیق، استفاده از تکنیک **کوانتزاسیون** است. هدف اصلی کوانتزاسیون، کاهش تعداد بیت‌های مورد نیاز برای نمایش پارامترهای مدل است. این کار معمولاً با تبدیل مقادیر اعشاری (اعداد با دقت بالا) به مقادیر صحیح (اعداد با دقت پایین‌تر) انجام می‌شود. در نتیجه، امکان فشرده‌سازی مدل فراهم شده و فضای حافظه مورد نیاز برای ذخیره‌سازی آن کاهش می‌یابد.

شایان ذکر است که کوانتزاسیون صرفاً به معنای گرد کردن اعداد به نزدیک‌ترین مقدار صحیح (رو به بالا یا پایین) نیست، بلکه فرآیندی سیستماتیک و مبتنی بر تحلیل آماری داده‌هاست که به گونه‌ای طراحی می‌شود تا کاهش دقت عددی، کمترین تأثیر را بر عملکرد مدل داشته باشد.

افزون بر کاهش فضای ذخیره‌سازی، کوانتزاسیون موجب **افزایش سرعت اجرای عملیات محاسباتی** نیز می‌شود. زیرا انجام محاسبات با اعداد صحیح به مراتب ساده‌تر، سریع‌تر و کم‌هزینه‌تر از محاسبه با اعداد اعشاری است، به‌ویژه بر روی سخت‌افزارهایی مانند پردازنده‌های گرافیکی[۹] یا پردازنده‌های خاص مانند TPU و DSP که برای این نوع بهینه‌سازی طراحی شده‌اند.

شبکه‌های عصبی می‌توانند از تعداد زیادی لایه‌های مختلف تشکیل شوند که هر یک نقش خاصی در فرآیند یادگیری و استخراج ویژگی ایفا می‌کنند. به عنوان مثال، لایه‌های خطی[۱۰] معمولاً از دو مؤلفه اصلی تشکیل شده‌اند :وزن‌ها[۱۱]و بایاس‌ها[۱۲] ، که اغلب به صورت اعداد اعشاری با دقت بالا (مانند اعداد ممیز شناور ۳۲ بیتی) نمایش داده می‌شوند.

---

[9] . Graphics processing unit
[10] . Linear Layers
[11] . Weights
[12] . Biases

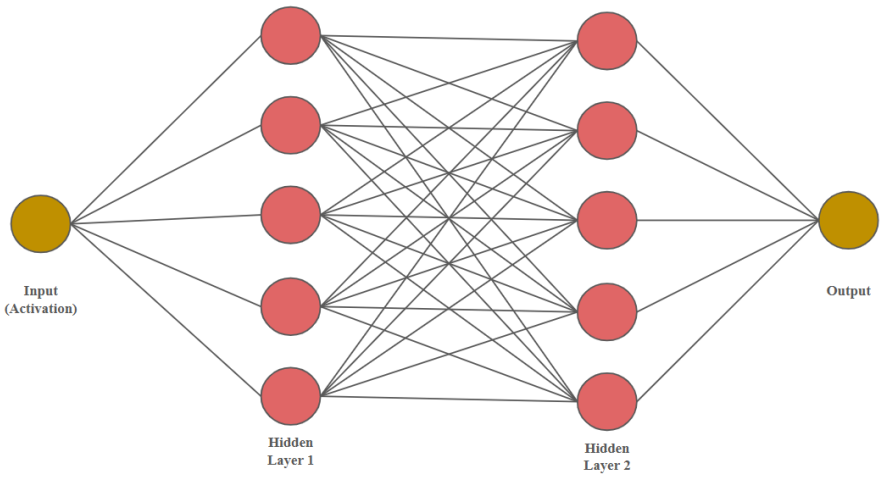

Figure 2 - Neural Network

$$X = Activation.Weight + Bias$$

هدف از کوانتزاسیون، کاهش دقت عددی این پارامترها و نمایش آن‌ها با اعداد صحیح (مثلاً ۸ بیتی) است، به گونه‌ای که عملکرد مدل به‌صورت معنادار کاهش نیابد .این فرآیند منجر به کاهش حجم مدل، صرفه‌جویی در مصرف حافظه و افزایش سرعت محاسباتی در حین استنتاج می‌شود، بدون آن‌که دقت عملکرد مدل به طور قابل توجهی افت کند.

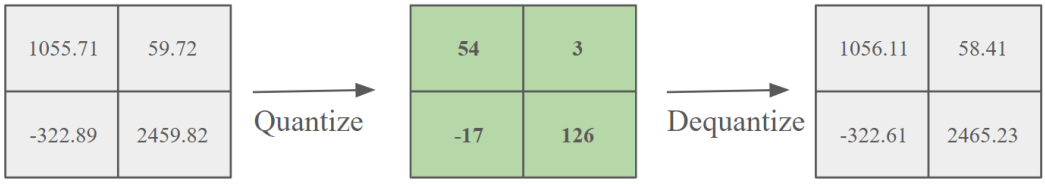

Figure 3 - Quantize to Dequantize

شکل ۳ یک نمونه ساده از فرآیند **کوانتزاسیون** و **دی‌کوانتزاسیون**[۱۳] را برای یک **ماتریس وزن ۲×۲** نمایش می‌دهد. در مرحله کوانتزاسیون، مقادیر اعشاری موجود در ماتریس اولیه به مقادیر صحیح در بازه اعداد ۸ بیتی تبدیل شده‌اند. سپس در مرحله دی‌کوانتزاسیون، این مقادیر صحیح مجدداً به مقادیر اعشاری بازگردانده شده‌اند. همان‌طور که مشاهده می‌شود، مقادیر بازسازی‌شده به مقادیر اصلی بسیار نزدیک هستند، هرچند با اندکی کاهش دقت عددی. این نمونه نشان می‌دهد که کوانتزاسیون می‌تواند با حفظ تقریبی دقت، منجر به کاهش حجم داده و ساده‌سازی محاسبات شود.

[13] . Dequantization

۱ .کوانتزاسیون متقارن[۱۴] و کوانتیزاسیون نامتقارن[۱۵]

کوانتیزاسیون متقارن

در روش متقارن، فرض بر آن است که داده‌ها به‌صورت متقارن حول مقدار صفر توزیع شده‌اند. در نتیجه، دامنه‌ی مقادیر عدد صحیح نیز به‌صورت متقارن و با مرکزیت صفر انتخاب می‌شود؛ برای مثال، در کوانتیزاسیون ۸ بیتی علامت‌دار، مقادیر بین[۱۲۷- ,۱۲۷+] قرار می‌گیرند. در این روش، تنها پارامتر مورد نیاز، ضریب مقیاس[۱۶] است که میزان فشرده‌سازی داده‌های اعشاری به فضای عدد صحیح را تعیین می‌کند. نقطه‌ی صفر[۱۷] همواره برابر با صفر در نظر گرفته می‌شود.

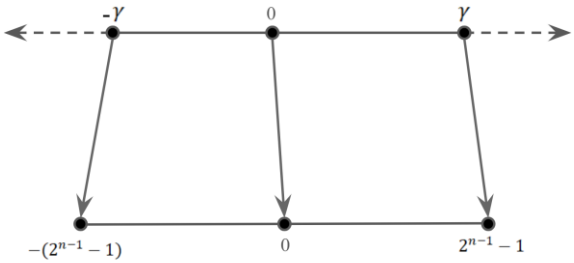

Figure 4 - Symmetric Quantization

فرمول کوانتیزاسیون متقارن به صورت زیر تعریف می‌شود:

$$q = round\left(\frac{x}{scale}\right), \qquad x = q \times scale$$

در این فرمول، x مقدار اصلی اعشاری و q مقدار کوانتیزه‌شده است. مقدار $scale$ معمولاً به صورت زیر محاسبه می‌شود:

$$scale = \frac{\max(|x|)}{127}$$

به عنوان نمونه، فرض کنید یک تنسور با مقادیر زیر در اختیار داریم:

$$[43.21, -44.93, 0, 22.99, -43.93, -11.35, 38.48, -20.49, -38.61, -28.02]$$

در این مجموعه، بیشینه مقدار مطلق برابر با ۴۴.۹۳ است. بنابراین، ضریب مقیاس به صورت زیر خواهد بود:

$$scale = \frac{44.93}{127} \approx 0.3538$$

با اعمال فرمول فوق، مقادیر کوانتیزه‌شده به شکل زیر حاصل می‌شوند:

$$[122, -127, 0, 65, -124, -32, 109, -58, -109, -79]$$

---

[14] . Symmetric Quantization
[15] . Asymmetric Quantization
[16] . Scale
[17] . Zero-point

کوانتیزاسیون نامتقارن

در مقابل، روش نامتقارن برای داده‌هایی طراحی شده است که توزیع آن‌ها حول صفر متقارن نیست و دارای انحراف[18] مشخصی هستند. در این روش، دامنه‌ی مقادیر عدد صحیح می‌تواند به صورت نامتقارن تعریف شود؛ برای نمونه، در کوانتیزاسیون ۸ بیتی بدون علامت، از بازه‌ی [۰, ۲۵۵] استفاده می‌شود. برخلاف روش متقارن، در اینجا دو پارامتر اصلی مورد نیاز است :ضریب مقیاس و نقطه صفر .نقطه صفر مقداری است که عدد صفر اعشاری در فضای عدد صحیح به آن نگاشته می‌شود و معمولاً مقدار عدد صحیحی بین ۰ تا ۲۵۵ است.

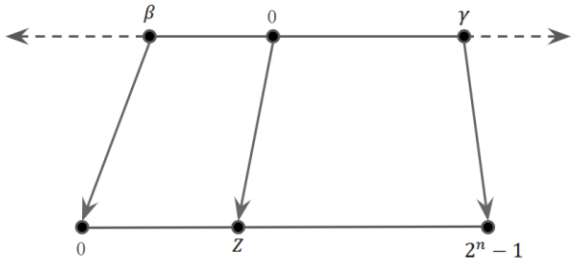

Figure 5 - Asymmetric Quantization

فرمول کوانتیزاسیون نامتقارن به صورت زیر تعریف می‌شود:

$$q = round(\frac{x - x_{min}}{scale}), \qquad x = q \times scale + x_{min}$$

که در آن، مقادیر x_{min} و x_{max} به ترتیب کمینه و بیشینه‌ی داده‌های اعشاری هستند، و ضریب مقیاس از رابطه‌ی زیر محاسبه می‌شود:

$$scale = (\frac{x_{max} - x_{min}}{255})$$

برای مثال قبل، داریم:

در نتیجه، با اعمال فرمول بالا، مقادیر کوانتیزه‌شده به صورت زیر به دست می‌آیند:

$$x_{min} = -44.93 \quad x_{max} = 43.21 \rightarrow scale = \frac{88.14}{255} \approx 0.3457$$

در اینجا مقدار صفر دقیقاً به عدد ۱۳۰ (zero-point) نگاشته شده است، و سایر مقادیر به تناسب موقعیت خود نسبت به بازه‌ی اصلی داده‌ها کوانتیزه شده‌اند.

$$[255, 0, 130, 196, 3, 97, 241, 71, 18, 49]$$

روش کوانتیزاسیون متقارن، به دلیل سادگی در پیاده‌سازی و عدم نیاز به نگهداریzero-point ، معمولاً در شرایطی به‌کار می‌رود که داده‌ها حول صفر توزیع متعادلی داشته باشند؛ مانند وزن‌های شبکه‌های عصبی. در مقابل، کوانتیزاسیون نامتقارن با

[18] . Bias

داشتن قابلیت تنظیمzero-point ، دقت بیشتری در بازنمایی داده‌هایی با بایاس یا توزیع نامتقارن فراهم می‌کند و غالباً برای کوانتیزاسیون خروجی لایه‌ها یا فعال‌سازی‌ها[19] ترجیح داده می‌شود.

۲ .کوانتزاسیون پس از آموزش[20]

در این روش، کوانتزاسیون پس از پایان آموزش مدل اعمال می‌شود. یعنی ابتدا مدل به‌صورت کامل با دقت بالا (مثلاً با پارامترهای ۳۲ بیت شناور) آموزش داده می‌شود و سپس در مرحله تبدیل به نسخه نهایی برای استقرا[21]ر ، پارامترها به نسخه‌ای با دقت پایین‌تر (مثلاً ۸ بیت صحیح) تبدیل می‌شوند.

در این تمرین، هدف پیاده‌سازی فرایند کوانتزاسیون نرم‌افزاری بر روی یک شبکه عصبی پیچشی با معماری LeNet-5 است. در این پروژه، از داده‌ی تصویری[22] Fashion-MNIST به عنوان مجموعه‌داده آموزشی و آزمایشی استفاده می‌شود. روش مورد استفاده برای کوانتزاسیون، کوانتزاسیون پس از آموزش است که طی آن، دقت عددی پارامترهای مدل کاهش می‌یابد (به عنوان مثال از ۳۲ بیت به ۸ بیت)، بدون نیاز به بازآموزی مدل. هدف از این فرایند، کاهش حجم مدل، تسهیل استقرار آن روی سخت‌افزارهای محدود، و افزایش سرعت استنتاج است، در حالی که افت دقت عملکرد مدل در حداقل ممکن باقی بماند.

---

[19] . Activations
[20] . Post-Training Quantization - PTQ
[21] . Deployment
[22] . https://github.com/zalandoresearch/fashion-mnist

پیاده سازی

در گام نخست، لازم است **کوانتزاسیون متقارن** و **نامتقارن** را به صورت دستی و **بدون استفاده از هیچ‌گونه کتابخانه آماده پیاده‌سازی کنید**. برای این منظور، از داده‌ای که در اختیار شما قرار گرفته استفاده نمایید. این فرایند باید به ترتیب برای نمایش پارامترها با دقت‌های عددی متفاوت تکرار شود:
ابتدا با استفاده از اعداد صحیح ۱۶ بیتی، سپس ۸ بیتی و در نهایت ۴ بیتی.

در گام دوم، لازم است با استفاده از **کتابخانه پای‌تورچ**[۲۳] فرآیند کوانتزاسیون نرم‌افزاری را بر روی مدل اعمال نمایید. پس از کوانت‌سازی، مدل طراحی‌شده که **دارای معماری LeNet-5** است، باید با استفاده از **مجموعه‌داده Fashion-MNIST** آموزش داده شود. در پایان مرحله آموزش، وزن‌های به‌دست‌آمده استخراج شده و به منظور استفاده در پیاده‌سازی سخت‌افزاری در فاز استنتاج مورد استفاده قرار خواهند گرفت.

برای بررسی تأثیر سطوح مختلف دقت عددی بر عملکرد مدل، آموزش و ارزیابی مدل باید با انواع مختلف نمایش عددی انجام شود، شامل:

- اعداد اعشاری ۳۲ بیتی(Floating Point 32-bit)
- اعداد اعشاری ۱۶ بیتی(Floating Point 16-bit)
- اعداد صحیح ۱۶ بیتی(Integer 16-bit)
- اعداد صحیح ۸ بیتی(Integer 8-bit)
- اعداد صحیح ۴ بیتی(Integer 4-bit)
- اعداد صحیح ۲ بیتی(Integer 2-bit)

[23] . PyTorch

در طراحی خود به نکات زیر توجه کنید:

ارزیابی

۱- پیاده‌سازی نرم‌افزاری

در این بخش به دنبال فهم دقیق الگوریتم‌های آموزش و استفاده اشاره شده در درس و محاسبات مربوط به این الگوریتم‌ها هستیم. با استفاده از زبان پایتون[۲۴] این شبکه را پیاده‌سازی کنید و برای داده‌های موجود آموزش دهید. نکات زیر را در نظر بگیرید:

گام اول: کوانتزاسیون متقارن و نامتقارن

- پیاده‌سازی کوانتزاسیون متقارن و نامتقارن به‌صورت دستی (بدون استفاده از کتابخانه‌های آماده)
- استفاده از کتابخانه NumPy مجاز است. (PyTorch و سایر فریم‌ورک‌ها مجاز نیستند)
- تکرار فرآیند کوانتزاسیون برای سطوح عددی مختلف:
  - اعداد صحیح ۱۶ بیتی
  - اعداد صحیح ۸ بیتی
  - اعداد صحیح ۴ بیتی
- ارزیابی مدل در هر سطح کوانتزاسیون
- رسم نمودار دقت در برابر سطح کوانتزاسیون[۲۵]
  - محور افقی :سطح عددی
  - محور عمودی :دقت مدل

گام دوم: کوانتزاسیون نرم‌افزاری با PyTorch

- استفاده از کتابخانه PyTorch برای اعمال کوانتزاسیون روی شبکه LeNet-5
- آموزش مدل با استفاده از دیتاست Fashion-MNIST
- اعمال کوانتزاسیون در سطوح عددی مختلف:
  - اعداد اعشاری ۳۲ بیتی
  - اعداد اعشاری ۱۶ بیتی
  - اعداد صحیح ۱۶ بیتی
  - اعداد صحیح ۸ بیتی
  - اعداد صحیح ۴ بیتی
  - اعداد صحیح ۲ بیتی
- استخراج وزن‌های نهایی جهت استفاده در پیاده‌سازی سخت‌افزاری
- رسم نمودار دقت در برابر بیت‌ویدث مشابه گام اول

---

[24]. Python
[25] . Bit-Width Quantization

مراجع:

[1] M. Horowitz, "Computing's energy problem (and what we can do about it)", *Proc. IEEE Int. Solid-State Circuits Conf. Dig. Tech. Papers*, pp. 10-14, 2014. doi: 10.1109/ISSCC.2014.6757323

[2] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, “A Survey of Quantization Methods for Effcient Neural Network Inference,” CoRR abs/2103 (2021): 13630.doi: 10.1201/9781003162810-13

توضیحات : خروجی‌های این تمرین شامل موارد زیر است:

فایل ارسالی یک فایل Zip است. خروجی‌ها برای هر سوال، در یک زیر شاخه‌ی جدا باشد. همه زیرشاخه‌ها در فایل Zip با نام HW1_StudentNumber_Fullname.zip گنجانده شود. در صورت درخواست زمان اضافی برای انجام تمرین، ابتدا به دستیار آموزشی در تلگرام پیام دهید و پس از هماهنگی، از ۴ روز کوپن خود استفاده کنید. چنانچه سوالی دارید می‌توانید از دستیار آموزشی درس در گروه تلگرام سوال نمایید.

لطفاً حتماً از قالب آماده گزارش که قبلاً تهیه شده است، استفاده کنید. هر گونه ارسال گزارش خارج از این قالب مورد بررسی قرار نخواهد گرفت.

موفق باشید.